Linear Algebra

HW1: Gaussian Elimination for a Linear Solver

Hong Shiang Lin

National Taipei University

hongshianglin@gm.ntpu.edu.tw

March 10, 2026

Tasks

- Proofs of Properties of Matrix Operations and Linear Combination (20%)
- Proofs of Properties of Linear Equation Systems (12%)
- Gaussian Elimination Practice (20%)
- A Linear Solver (20%)
- Report (20%)

Rules for Proofs

- Use only theorems or properties from pages **before** the one where the theorem appears. Do not cite the same page or later pages.
- Present proofs in enumerated steps, each limited to **two sentences**.
- Follow the **mathematical symbol conventions** of this course.
- Write a brief explanation of your reasoning **below your proof**.

Proofs of Properties of Matrix Operations and Linear Combination

Q1: Matrix Operations (16%)

- $A + B = B + A$
- $A + (-A) = O$
- $(st)A = s(tA)$
- $(A^T)^T = A$
- $A + O = A$
- $s(A + B) = sA + sB$
- $(s + t)A = sA + tA$
- $(sA)^T = sA^T$

Grading Criteria

- Level of Justification

Q2: Linear Combination (14%)

- $A(c\mathbf{u}) = (cA)\mathbf{u}$ for every scalar c.
- $(A + B)\mathbf{u} = A\mathbf{u} + B\mathbf{u}$.
- $A\mathbf{e}_j = \mathbf{a}_j$ for j =1, 2, ..., n, where $\mathbf{e}_j$ is the jth standard vector in $\mathbb{R}^n$.
- If B is an $m \times n$ matrix such that $B\mathbf{w} = A\mathbf{w}$ for all $\mathbf{w}$ in R^n, then $B = A$
- $A\mathbf{0}$ is the $m \times 1$ zero vector, where $\mathbf{0}$ is the $n \times 1$ zero vector.
- $O\mathbf{v}$ is the $m \times 1$ zero vector
- $I_n\mathbf{v} = \mathbf{v}$

Grading Criteria

- Level of Justification

Proofs of Properties of Linear Equation Systems

Q1: Rank and Nullity (4%)

Prove that the rank of a matrix equals the number of pivot columns in the matrix according to the definitions of rank and pivot columns.

Grading Criteria

- Level of Justification

Q2: Test for Consistency (6%)

Prove that the following two conditions are equivalent.

- The matrix equation $A\mathbf{x} = \mathbf{b}$ is consistent.
- The vector $\mathbf{b}$ is a linear combination of the columns of A.

Grading Criteria

- Level of Justification

Gaussian Elimination Practice

(Apply Gaussian elimination techniques (steps presented on slides) to derive solutions for the linear systems provided in the earlier examples, which are listed again below.)

Q1: Leontief Input/Output Model (6%)

$$(I_3 - C)\mathbf{x} = \mathbf{d}, C = \begin{bmatrix} 0.1 & 0.2 & 0.1 \\ 0.2 & 0.4 & 0.2 \\ 0.3 & 0.1 & 0.1 \end{bmatrix}, \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \mathbf{d} = \begin{bmatrix} 90 \\ 80 \\ 60 \end{bmatrix}$$

Describe your gaussian elimination procedure to derive the following solution of the system.

$$\begin{bmatrix} 170 \\ 240 \\ 150 \end{bmatrix}$$

Grading Criteria

- Correctness (Answer and Process)
- Level of Detail

Q2: Electrical Circuit (4%)

$$\begin{array}{rcrcrcr} 2I_1 & - & 4I_2 & & & = & 0 \\ & & 4I_2 & + & 2I_3 & = & 30 \\ I_1 & + & I_2 & - & I_3 & = & 0 \end{array}$$

Describe your gaussian elimination procedure to derive the solution of the system: $I_1 = 6$, $I_2 = 3$, and $I_3 = 9$.

Grading Criteria

- Correctness (Answer and Process)
- Level of Detail

A Linear Solver

This is a program homework to help us to be familiar with matrix-vector operations, Gaussian Elimination, and systems of linear equations.

You should complete the following codes:

- *matrices_vectors.py*: Basic matrix and vector operations.
- *gauss.py*: Elementary row operations and Gaussian Elimination.
- *linear_solver.py*: Use Gaussian Elimination to solve linear equations.
- *test_linear_solver.py*: Test the libraries.

(Introduction to *test_linear_solver.py*)

```
import torch
from linear_solver import solve_linear_equations
from matrices_vectors import matrix_vector_product, matrix_sum, scalar_
    matrix
```

- Required libraries.

notes¡all¿

```python
if __name__ == "__main__":
    run_test(A_consistent, b_consistent, "Consistent Case")
    run_test(A_inconsistent, b_inconsistent, "Inconsistent Case")
```

- Run two tests: a consistent system and an inconsistent system.

notes¡all¿

```python
# 1. Define Test Data (4 equations, 4 unknowns)
# Case 1: Consistent System (Unique solution or Infinite)
A_consistent = torch.tensor([
    [1, 2, 1, -1],
    [3, 2, 4,  4],
    [4, 4, 3,  4],
    [2, 0, 1,  5]
], dtype=torch.float32)

b_consistent = torch.tensor([[5], [16], [22], [15]], dtype=torch.
    float32)
```

- Consistent case: Unique solution.

notes¡all¿

```python
# Case 2: Inconsistent System
# (Row 3 is a multiple of Row 1, but b is not)
A_inconsistent = torch.tensor([
    [1, 1, 1, 1],
    [2, 3, 1, 4],
    [1, 1, 1, 1], # Same coefficients as row 1
    [0, 1, 2, 3]
], dtype=torch.float32)

b_inconsistent = torch.tensor([[10], [20], [5], [15]], dtype=torch.
    float32) # b[2] contradicts b[0]
```

- Inconsistent case: No solution.

notes¡all¿

```python
def run_test(A, b, test_name):
    print(f"--- Testing: {test_name} ---")
    x = solve_linear_equations(A, b)
    if x is None:
        print("Result: No Solution (Inconsistent)\n")
    else:
        print("Result: Solution Vector x:")
        print(x)
        # TODO: Calculate Error = Ax - b (it should be close to zero)
        # Hint: Use our custom matrix_vector_product to compute the
            linear combination: Ax
        # Hint: Use our custom scalar_matrix and matrix_sum to get the
            error vector: Ax-b
        # Hint: Get the L2 Norm (Euclidean distance) of the error
            vector (you may use torch.norm)
        # Hint: Report the L2 Norm result
```

- Implementation of *run_test*: Used our custom function *solve_linear_equations*
- TODO: Use functions in matrices vectors to implement the error of solution vectors.
- TODO: Report errors (L2 norm).

Grading Criteria

- Correctness.

notes¡all¿

```
# BONUS: System Robustness & Scalability Challenge
# Consider to define the following cases and report the results.
#
# 1. Minimal Case (2x2 or 3x3):
#    - Test a simple, manually verifiable system to ensure core logic
    is sound.
# 2. Non-Square (Rectangular) Matrices:
#    - Underdetermined: "Fat" matrix (e.g., 3x5). Does it handle free
    variables?
#    - Overdetermined: "Tall" matrix (e.g., 6x3). Test for both
    consistent
#      and inconsistent cases.
# 3. Large Scale (100x100+):
#    - Generate a large matrix A and vector x_true.
# 4. Report Requirements:
#    - List the dimensions and properties of your test data.
#    - Provide the final verification error ||Ax - b|| for each case.
```

- BONUS: Define more cases to test your system.

Grading Criteria

- Representativeness
- Richness

notes¡all¿

(Introduction to *linear_solver.py*)

```
def solve_linear_equations(A, b):
    # Create Augmented Matrix [A | b]
    augmented_matrix = torch.cat((A, b), dim=1)

    rows, cols_aug = augmented_matrix.shape
    num_vars = cols_aug - 1
    solution = torch.zeros((num_vars, 1), dtype=torch.float32) #
        Solution is default to zero

    # TODO: Use our custom gauss_elimination to obtain the Reduced Row
        Echelon Form.
    # Then, use test_consistency to inspect the reduced matrix.
    # Return None if the system is inconsistent. Otherwise, return the
        results of generate_solution.
    return solution
```

- Implementation of *solve_linear_equations*.
- TODO: Use custom functions to determine the solution.

Grading Criteria

- Correctness

notes¡all¿

```python
def test_consistency(augmented_RREF):
    """
    Checks if the system Ax = b is consistent.
    """
    rows, cols = augmented_RREF.shape
    # TODO: Use the following property to determine consistency:
    # A system is inconsistent if and only if a row in the RREF looks
        like [0 0 ... 0 | b_i]
    # where b_i != 0.
    # Return False if inconsistent

    return True # Consistent
```

- Implementation of *test_consistency*.
- TODO: Determine consistency based on the specified property.

Grading Criteria

- Correctness

notes¡all¿

```python
def generate_solution(augmented_RREF):
    """
    Extracts a solution vector. Basic variables are solved, Free
        variables are set to 1.
    """
    rows, cols_aug = augmented_RREF.shape
    num_vars = cols_aug - 1
    solution = torch.ones((num_vars, 1), dtype=torch.float32) # Default
         free variables to 1

    # TODO: Identify pivot columns
    # Hint: Find the first nonzero in each row

    # TODO: Calculate basic variables
    # Hint: Note that we have assume all free variables are set to 1

    return solution
```

- Implementation of *generate_solution*.
- TODO: Through pivot columns to identify basic variables and free variables, then determine a unique solution.

Grading Criteria

- Correctness

notes¡all¿

(Introduction to *gauss.py*)

```
def gauss_elimination(A):
    """
    Transforms matrix A into RREF using the encapsulated row operations
        .
    """
    R = A.clone().to(torch.float32)
    rows, cols = R.shape

    pivot_row = 0
    pivot_col = 0
```

notes¡all¿

```
    # Set zero thresh (if entry is larger than this value, treat it as
        zero. Otherwise, treat it as nonzero)
    zero_thresh = 1e-6

    # TODO: Implement step 1 to step 4 in the lecture slides (the
        forward phase to Row Echelon Form)
    # Hint: You should use our custom row_interchange, row_scaling, row
        _addition funtions

    # TODO: Implement step 5 and step 6 in the lecture slides (the
        backward phase to Reduced Row Echelon Form)
    # Hint: You should use our custom row_interchange, row_scaling, row
        _addition funtions

    return R
```

- Implementation of *gauss_elimination*.
- TODO: Use custom row operations to implement the forward phase to Row Echelon Form.
- TODO: Use custom row operations to implement the backward phase to Reduced Row Echelon Form.

Grading Criteria

- Correctness

notes¡all¿

```python
def row_interchange(R, i, j):
    """Interchanges row i and row j of matrix R."""
    # We use clone() to ensure we don't have reference issues during swapping
    temp = R[i].clone()
    # TODO: Perform row interchange
    return R
```

- Implementation of *row_interchange*.
- TODO: Implement the row interchange based on the definition in lecture slides.

Grading Criteria

- Correctness

notes¡all¿

```python
def row_scaling(R, i, s):
    """Scales row i of matrix R by a scalar s."""
    cols = R.shape[1]
    # TODO: Use our custom scalar_matrix function to finish this row operation
    # Hint: To use the scalar_matrix, a row of 1D tensor should be reshaped to a row 2D tensor.
    # For example, R[i] should be reshaped as (1, cols) to fit the function requirments
    # Hint: You may use API flatten() to convert a 2D tensor back to 1D tensor
    return R
```

- Implementation of *row_scaling*.
- TODO: Use custom matrix operations to implement the row scaling.

Grading Criteria

- Correctness

notes¡all¿

```python
def row_addition(R, i, j, s):
    """Adds s times row i to row j."""
    cols = R.shape[1]
    # TODO: Use our custom scalar_matrix function and matrix_sum to finish this row operation
    # Hint: To use the scalar_matrix and matrix_sum, a row of 1D tensor should be reshaped to a row
    # 2D tensor.
    # For example, R[i] should be reshaped as (1, cols) to fit the function requirments
    # Hint: You may use API flatten() to convert a 2D tensor back to 1D tensor
    return R
```

- Implementation of *row_addition*.
- TODO: Use custom matrix operations to implement the row addition.

Grading Criteria

- Correctness

notes¡all¿

(Introduction to *matrices_vectors.py*)

```
def scalar_matrix(s, M):
    rows = M.shape[0]
    cols = M.shape[1]
    # Initialize a tensor of zeros with the same shape as M
    sM = torch.zeros(M.shape, dtype=torch.float32)
    # TODO: Complete the functionality by incorporating a for loop to
        scale each entry
    return sM
def matrix_sum(M1, M2):
    rows = M1.shape[0]
    cols = M1.shape[1]
    # Initialize a tensor of zeros with the same shape as M1
    M = torch.zeros(M1.shape, dtype=torch.float32)
    # TODO: Complete the functionality by incorporating a for loop to
    # add corresponding entries of M1 and M2 in a general manner.
    return M
```

- Implementation of *scalar_matrix* and *matrix_sum*.
- TODO: Complete scalar matrix multiplication and matrix summation with entry-level implementation (for loops).

Grading Criteria

- Correctness

notes¡all¿

(Introduction to *matrices_vectors.py*)

```
def matrix_vector_product(M, vec):
    rows, cols = M.shape[0], M.shape[1]
    vec2 = torch.zeros((rows, 1), dtype=torch.float32)

    # TODO: Complete the functionality by implementing a for loop for a
        general linear combination.
    # Hint: Utilize the scalar_matrix() and matrix_sum() functions.
    return vec2
```

- Implementation of *matrix_vector_product* and *matrix_sum*.
- TODO: Complete matrix vector product by using custom matrix operations.

Grading Criteria

- Correctness

notes¡all¿

Report

Content

- Theory (2%)
- Homework Requirements (2%)
- Questions (2%)
- Work Division (2%)
- Progress Plan (2%)
- Answers of Conceptual Problems
- Code Snippets of TODOs
- Meeting Notes (2%)
- Worktime (2%)
- Reflection (2%)

Appearance

- Appropriateness of formatting/layout (2%)
- Consistency of writing style (2%)

Contribution

Contribution of each member:

- If one's contribution is obviously identified lower than others, he/she gets lower scores.
- If one's name does not appear on the report, it will be regarded that this one has no contribution.

Theory

Assignment

- List the **key definitions, theorems, and properties** from the course.
- **Explain their logical relationships** clearly.
- Create **one mind map** showing how these concepts are connected.

Grading Criteria

1. **Completeness** – Coverage of important course content
2. **Rationality** – Clarity and soundness of logic
3. **Preciseness** – Accuracy and precision of explanations

Homework Requirements

Assignment

- Create a **table** that summarizes the **assignment requirements** you have identified.
- For each requirement, provide a brief **abstract** explaining your understanding of what needs to be done.
- This assignment should be done before implementing homework.

Grading Criteria

1. **Completeness** – Coverage of homework content
2. **Preciseness** – Clarity and accuracy of the descriptions

Questions

Assignment

- List the **questions or issues** you encounter regarding the **theory**, **conceptual problems**, and the **code skeleton**.
- For each question or issue, present your **conclusion before consulting AI or teacher**.
- Then provide the **conclusion obtained after consulting AI or teacher**.
- This assignment should be done before implementing homework.

Grading Criteria

1. **Richness** – Variety and depth of the questions or issues raised
2. **Preciseness** – Clarity and accuracy of the descriptions

Progress Plan

Assignment

- Create a **table** showing your **weekly progress timelines**.
- For each week, provide a brief **explanation of the planned tasks**.
- Include items that are not explicitly listed in the assignment requirements, such as **reviewer tasks**.
- Note: this assignment should be done before implementing the homework.

Grading Criteria

1. **Rationality** – Logical sequencing of tasks in the timelines
2. **Completeness** – Inclusion of all relevant tasks

Work Division

Assignment

- List the **work assignments** within your group.
- Explain the **reasoning behind the role distribution** and why each member was assigned their tasks.

Grading Criteria

1. **Clarity** – How clearly the work assignments and rationale are described
2. **Completeness** – How comprehensively the tasks and roles are covered

Roles on the Work

- **Implementer**: The one who answer a question or write a piece of program.
- **Reviewer**: The one who examine an answer or inspect/test a piece of program.
-
- Each subtask must have at least one implementer and one reviewer; they must be different people.
- The roles of implementer or reviewer for different problems or code segments can be taken by different ones by exchanging.

Answers of Conceptual Problems

Assignment

- For each **conceptual problem**, include the **original question**.
- Provide the **solution submitted by the implementer**.
- Include the **reviewer's feedback**, which must at least **Approve** the solution. You may also include any insights or lessons you learned from the implementer.

Grading Criteria

1. See the grading instructions for each individual problem.

Code Snippets of TODOs

Assignment

- List all **code snippets** corresponding to TODOs.
- For each code block, provide a **paragraph of explanatory text** below it.
- The explanatory text should describe the **logic or grammar of the code** as well as the **motivation or purpose** behind it.

Grading Criteria

1. See the grading instructions for each individual TODO.
2. **Completeness** – How well the code's logic or grammar is explained
3. **Understanding** – Clarity and depth in explaining the motivation or purpose

Meeting Notes

Assignment

- Summarize the **issues discussed** between the implementer and the reviewer.
- For each issue, provide a concise **summary of its content** along with the **conclusions reached** at the time.

Grading Criteria

1. **Richness** – Variety and depth of the issues discussed
2. **Preciseness** – Clarity and accuracy of the summaries

Worktime

Assignment

- List the **amount of time** spent on each work item.
- Provide the **total time** spent on the assignment.
- For any item that took an unusually long time, explain the **specific points or difficulties** encountered.

Grading Criteria

1. **Completeness** – All work items and total time are reported
2. **Preciseness** – Explanations of difficulties are clear and specific

Reflection

Assignment

- Describe **what you have learned** from this assignment.
- Reflect on your **individual learning style** when facing problems.

Grading Criteria

1. **Concreteness** – How specific and tangible the learning points are
2. **Preciseness** – Clarity and accuracy in explaining difficulties
3. **Depth** – Insightfulness and thoughtful analysis in reflecting on your learning style

Submission Guidelines

- Write members' names on the top of the report.
- Put all the codes into a folder *codes*.
- Named the report with *report.pdf*
- Put **codes** and *report.pdf* into a folder **hw1**.
- Compress **hw1** and specify the file format as hw1.zip.
- Upload hw1.zip onto Digital Platform 3.
- Deadline: 12:00 a.m. on March 25.

Unsatisfied submission will cause 5 point subtraction on the HW score.

The End

Questions? Comments?

Origin of The Latex Template

The latex template is downloaded from:
https://www.LaTeXTemplates.com

The Author: Vel (vel@latextemplates.com)